

AI时代的PRD（输入）规范-副本

正文 1082 字 (1 页) | 浏览 1 次 (1 人) | 全部 >

AI 速览

prd准入标准门禁 Skill: prd-readiness-gate (prd-readiness-gate)

PRD = 在已有边界下的显式上下文 + 逻辑闭环 + 可自动化验证的断言

它依然是 PRD，但它是“AI 时代”特有的 PRD。

你之所以觉得它像技术方案，是因为在人类开发时代，“业务（What）”和“技术（How）”之间有一道由人类工程师的经验、常识和设计能力筑起的“防火墙”。

但在 AI AutoPilot 的语境下，这道防火墙倒塌了。我们必须重新定义 PRD 与技术方案的边界：

传统研发 vs AI AutoPilot 的职责推演

我们可以通过下面这个简单的对比，看看为什么 PRD 必须向技术侧“左移”：

阶段	传统研发模式（面向人类）	AutoPilot 模式（面向 AI）
产品经理（PRD）	业务逻辑（What）： 只管业务流程和页面长什么样。剩下的留给开发去猜、去问、去设计。	业务逻辑 + 确定性上下文： 必须把业务逻辑的“边界”和“输入输出契约”死死框住，不给 AI 留瞎猜的空间。
AI 托管开发（AutoPilot）	（由人类开发承接）	执行与代码组装（How）： 专注于具体的代码怎么写、怎么设计类、怎么调用底层 API、怎么实现算法。
技术方案（设计文档）	架构设计（How）： 数据库表设计、系统微服务拓扑、技术选型。	系统架构拓扑与规范： 依然存在，但它变成了 Harness（围栏）的一部分，用来约束 AI 的行为。

为什么说“显式上下文 + 状态机 + 自动化断言”依然是 PRD？

我们拆开来看，这三个核心要素并没有剥夺开发（AI）去写代码和设计架构的权利，它们只是**极其精准的业务边界定义**：

1. 显式技术上下文不是写代码

- **技术方案写的是：**“我们需要建一个表，字段 A 用 BigInt，字段 B 加唯一索引，并且采用 Redis 分布式锁来防止并发。”
- **AI 时代的 PRD 写的是：**“本需求只涉及餐饮系统的『影子账本（Shadow Account）』模块，涉及的核心业务实体是**商户流水**。开发时必须引用已有的商户账户上下文，不能新建无关的账户模型。”
- **本质：**这是在给 AI 划定**业务操作的范围**，而不是教它怎么写代码。

2. 状态机闭环逻辑 不是 伪代码

- **技术方案写的是：**“在 Controller 层拦截请求，开启事务，如果 A 失败则回滚，B 成功则调用通知服务。”
- **AI 时代的 PRD 写的是：**“当账单状态为 **待对账** 时，仅允许转为 **已对账** 或 **对账异常**。严禁从 **待对账** 直接跳过对账流程变成 **已结算**。”
- **本质：**状态机是**纯粹的业务规则**。传统 PM 喜欢用文字叙述，导致 AI 漏掉分支；AI 时代的 PRD 要求用矩阵或图表把业务分支封死。

3. 可自动化验证的断言 不是 单元测试代码

- **技术方案写的是：**“使用 Mockito 模拟数据库连接，编写 JUnit Test Case，断言 `assertEqual(100, balance)`。”
- **AI 时代的 PRD 写的是：**“用自然语言或 Gherkin 语法写明验收标准：**当商户余额为 0 时，扣款应当失败，并提示余额不足。**”
- **本质：**需求本身就应该定义“什么叫做出厂合格”。PM 只定义**业务测试用例的数据输入输出 (I/O)**，至于怎么用代码去跑这个测试，是由 AI 自己或者 Harness 系统去完成的。

总结

你可以这样理解：

在 AI 时代，PRD 的本质变成了“**业务代码的编译器配置文件**”。

如果 PRD 依然像以前那样只写一句话——“**做一个类似美团的外卖商户对账功能**”，AI 绝对会写出一堆无法运行的垃圾代码。

你现在要确立的规范，不是让 PM 去抢开发的饭碗写技术方案，而是**逼着 PM 把过去由于偷懒、依赖口头沟通而省略掉的“业务细节和确定性边界”，在需求阶段就彻底显式化**。只有这样，AutoPilot 的齿轮才能真正咬合转动。

Ⓔ 仅供内部使用，未经授权，切勿外传